

Agathiyam: Sandhi-Aware Tokenization for Tamil

Roshini Priya R¹, Subika M², Venkatapathy S³

^{1,2} PSG Institute of Technology and Applied Research, Coimbatore, India

³ Independent Researcher, Coimbatore, India

{roshinipriya510, subikamaha}@gmail.com

Abstract

Tokenization plays a critical role in Natural Language Processing (NLP) pipelines, directly affecting performance in language modeling, machine translation, and other downstream tasks. For morphologically rich and agglutinative languages like Tamil, existing tokenizers such as Byte Pair Encoding (BPE) often fail to handle complex morphophonemic transformations caused by Sandhi rules, resulting in semantically lossy segmentations.

In this work, we propose Agathiyam, a Sandhi-aware tokenization framework designed for Tamil. Our method integrates linguistically curated Sandhi rules with grapheme-aware pre-segmentation and Constrained Byte Pair Encoding (CBPE) to preserve phonological structure. We conduct extensive experiments on the Samantar parallel dataset for Tamil-English and Tamil-Hindi translation tasks, demonstrating improvements in segmentation accuracy, token efficiency, and downstream BLEU scores compared to standard BPE/GPE baselines. For evaluation and benchmarking only, we also compare against byte-level models such as google/byt5-base; no ByT5 fine-tuning is used in our pipeline.

1 Introduction

Tokenization forms the foundation of any NLP system, determining how raw text is segmented into units such as characters, subwords, or words. Its effectiveness directly impacts vocabulary compactness, semantic preservation, and the efficiency of language models. While English and other morphologically simple languages benefit from frequency-driven approaches like BPE, Tamil presents unique challenges.

Tamil is an agglutinative Dravidian language where morphemes frequently combine using phonological transformations governed by Sandhi rules. These transformations modify vowels, consonants, or syllable structures when words are combined, leading to ambiguity in segmentation. A purely statistical approach like BPE often results in incorrect or meaningless subwords.

To address this, we propose Agathiyam, a Sandhi-aware tokenization framework that integrates rule-based linguistic knowledge with grapheme-aware statistical optimization. We evaluate our approach on the Samantar dataset, a large-scale multilingual parallel corpus, demonstrating improvements in tokenization efficiency and translation quality.

2 Related Work

Tokenization techniques can be broadly categorized into statistical, rule-based, and hybrid methods:

Table 1: Comparison of Tokenization Approaches

Approach	Method	Limitations
Byte Pair Encoding (BPE)	Greedy frequency-based subword merges	Produces meaningless subwords for Sandhi-heavy Tamil words
Grapheme Pair Encoding (GPE)	Grapheme-level merges preserve orthographic structure	Improves character coverage but fails on compound Sandhi rules
Morphological Tokenization	Uses linguistically curated morphemes for segmentation	High accuracy but dependent on manual annotation
Agathiyam (Proposed)	Sandhi-aware pre-splitting + grapheme segmentation + constrained BPE	Handles unseen forms via rules, preserves phonology, improves efficiency

While previous work focuses primarily on statistical segmentation, our framework integrates linguistic expertise with grapheme-aware modeling to handle Tamil-specific morphological phenomena effectively.

3 Methodology

Our proposed methodology combines three components:

- Sandhi-aware Pre-tokenization: Apply linguistically curated Sandhi rules to segment words.
- Grapheme Segmentation: Split each Sandhi segment into Unicode grapheme clusters.
- Constrained BPE (CBPE): Train BPE merges over grapheme IDs while preserving dependent vowel attachments.

3.1 Pre-tokenization

Statistical tokenizers like BPE often split morphologically meaningful Tamil words incorrectly. For example:

- Word:
- Correct segmentation: + +
- BPE segmentation: + + + (incorrect)

This motivates the need for linguistically informed segmentation.

3.1.1 Sandhi-aware Lookup Pre-tokenization

We constructed a Sandhi-aware lookup using:

1. Automatic generation via Tamil Sandhi rules, suffix lists, and common inflectional patterns.
2. Manual refinement with linguistic experts for morphological correctness.

Table 2: Sample Tamil Sandhi-aware Lookup Table

Word	Segments
	++ ++ + +

3.2 Handling Sandhi Rules in Tamil

Tamil Sandhi rules are integrated into the pre-tokenization stage. We categorize them as:

1. Vowel Coalescence: $+ \rightarrow$
2. Consonant Assimilation: $+ \rightarrow$
3. Glide Insertion: $+ \rightarrow$
4. Deletion Rules: $+ \rightarrow$

3.3 Sandhi-Grapheme-BPE Training Pipeline

Algorithm 1 Sandhi-Grapheme-BPE (Sandhi-GPE) Tokenization Pipeline

Require: Samanantar Tamil corpus C , number of merges K

Ensure: Vocabulary V , merge rules M , tokenizer T

- 1: Initialize $V \leftarrow \emptyset$, $M \leftarrow \emptyset$
 - 2: for each sentence $s \in C$ do
 - 3: $S \leftarrow \text{SandhiSplit}(s)$ // split using Tamil Sandhi rules
 - 4: for each token $t \in S$ do
 - 5: $G \leftarrow \text{GraphemeClusterSplit}(t)$
 - 6: Add each grapheme in G to V
 - 7: end for
 - 8: end for
 - 9: Encode corpus as sequences of grapheme IDs using V
 - 10: for $i = 1 \dots K$ do
 - 11: $p \leftarrow$ most frequent adjacent ID pair in encoded corpus
 - 12: Merge p into a new symbol v ; update encoded corpus
 - 13: Update $V \leftarrow V \cup \{v\}$ and $M \leftarrow M \cup \{p \mapsto v\}$
 - 14: end for
 - 15: Save artifacts (V, M)
 - 16: Construct tokenizer $T \leftarrow \text{SandhiBPETokenizer}(V, M)$
 - 17: return V, M, T
-

3.4 Constrained BPE for Tamil

Tamil vowels and diacritics often get split incorrectly by naive BPE. Our Constrained BPE (CBPE) attaches dependent vowels to their preceding base consonants during initialization, ensuring phonological consistency.

Algorithm 2 Constrained BPE (CBPE) Initialization

Require: Grapheme inventory G

Ensure: Seed vocabulary V_0

- 1: $V_0 \leftarrow$ combine base consonants with dependent vowels (attach diacritics to bases)
 - 2: return V_0
-

4 Experiments

We evaluate Agathiyam using the Samantar multilingual parallel dataset, which contains Tamil-English and Tamil-Hindi sentence pairs. We compare against BPE and GPE baselines. In addition, we report comparisons against the byte-level model google/byt5-base strictly as an evaluation baseline (compression ratio, parity); it is not trained or integrated into our tokenizer.

4.1 Dataset Details

Table 3: Samantar Dataset Statistics

Language Pair	# Sentences	# Word-Segment Pairs
Tamil-English	57,842	211,093
Tamil-Hindi	52,416	193,505

4.2 Evaluation Metrics

We adopt both intrinsic and extrinsic evaluation:

- Segmentation Accuracy – Measures correctness of token splits.
- Fertility Score – Average number of tokens per word (lower is better).
- Compression Ratio & Parity – For comparisons with byte-level baselines (e.g., ByT5).

4.3 Results

To demonstrate the effectiveness of our Sandhi-aware Grapheme Pair Encoding (Sandhi-GPE), we compare it against standard BPE and GPE tokenization on the sentence:

4.3.1 Tokenization Comparison

Table 4: Comparison of BPE, GPE, and Sandhi-GPE tokenization outputs

Method	Tokens	# Tokens
BPE	[, , , , , , , □, , , , , , , , □, , , , .]	24
GPE	[, , , □, , , □, , , , , , □, , , .]	16
Sandhi-GPE	[, , , .]	4

4.3.2 Decoded Sentences

- BPE Decoded: .
- GPE Decoded: .
- Sandhi-GPE Decoded: .

4.3.3 Analysis

- BPE produces excessive splits (24 tokens), breaking meaningful morphemes.
- GPE improves orthographic grouping (16 tokens) but still lacks semantic awareness.
- Sandhi-GPE captures morphological units effectively, yielding compact and semantically faithful segmentation (4 tokens).